

Assignment 07

For this assignment you will be writing a series of programs that should be stored in their own ".py" files. The filenames you should use are listed at the end of each part. When you're finished you should submit your program to the Assignment #7 category inside of Brightspace.

Problem 0: New User

You have been asked to write a username validation program for a small website. The website has specific rules on what constitutes a valid username, including:

- All usernames must be between 8 and 15 characters long
- Usernames can only contain alphabetic (a-z and A-Z) and numeric characters (0-9) - no special characters or spaces are allowed.
- The first and last characters in a username cannot be a digit
- Usernames must contain at least one uppercase character
- Usernames must contain at least one lowercase character
- Usernames must contain at least one numeric character

Write a program that asks the user to enter in a username and then examines that username to make sure it complies with the rules above. Here's a sample running of the program - note that you want to keep prompting the user until they supply you with a valid username:

```
Enter a username: craig
* Length of username: 5
* All characters are alpha-numeric: True
* First & last characters are not digits: True
* # of uppercase characters in the username: 0
* # of lowercase characters in the username: 5
* # of digits in the username: 0
Username is invalid, please try again

Enter a username: craig123
* Length of username: 8
* All characters are alpha-numeric: True
* First & last characters are not digits: False
* # of uppercase characters in the username: 0
* # of lowercase characters in the username: 5
* # of digits in the username: 3
Username is invalid, please try again
```

Enter a username: craig 123

- * Length of username: 9
- * All characters are alpha-numeric: False
- * First & last characters are not digits: False
- * # of uppercase characters in the username: 0
- * # of lowercase characters in the username: 5
- * # of digits in the username: 3

Username is invalid, please try again

Enter a username: Craig123

- * Length of username: 8
- * All characters are alpha-numeric: True
- * First & last characters are not digits: False
- * # of uppercase characters in the username: 1
- * # of lowercase characters in the username: 4
- * # of digits in the username: 3

Username is invalid, please try again

Enter a username: Craig123!

- * Length of username: 9
- * All characters are alpha-numeric: False
- * First & last characters are not digits: True
- * # of uppercase characters in the username: 1
- * # of lowercase characters in the username: 4
- * # of digits in the username: 3

Username is invalid, please try again

Enter a username: Craig123T

- * Length of username: 9
- * All characters are alpha-numeric: True
- * First & last characters are not digits: True
- * # of uppercase characters in the username: 2
- * # of lowercase characters in the username: 4

```
* # of digits in the username: 3
```

```
Username is valid!
```

Hint: you will need to count the # of uppercase, lowercase and digit characters using some kind of loop.

This program should be named as follows: LastNameFirstName_assign7_part0.py

Problem 0 Points (25):

Each of the following 5 test cases produce the following outputs:

Input	Length	Alpha-num only	First/last not digit	# Upper	# Lower	# Digits	Valid
1amanda2	8	True	False	0	6	2	False
Amanda@43	9	False	False	1	5	2	False
Ama2nda	7	True	True	1	5	1	False
AmandalsCool	12	True	True	3	9	0	False
Amanda95S	9	True	True	2	5	2	True

Problem 1: Telephone Numbers

For this problem, you will be asking the user for a phone number and formatting it. For the purposes of this problem, you can assume that all phone numbers are in the United States (country code = +1). You can also assume that the user will only enter numbers. The phone number should have an area code and therefore be 10 digits long (area code is 3 digits, followed by the phone number itself, which is 7 digits). You must validate this. A formatted phone number, including country code and area code, looks like this (added characters are highlighted):

+1 (123) 456-7890

Here is a sample running of this program:

```
What is your phone number? 5556789
Phone number is too short, please try again!
What is your phone number? 123555678900
Phone number is too long, please try again!
What is your phone number? 1235556789
Your phone number is +1 (123) 555-6789.
```

This program should be named as follows: LastNameFirstName_assign7_part1.py

Problem 1 Points (18):

- Loop does not allow phone number of length < 10 (3 points)
- Loop does not allow phone number of length > 10 (3 points)
- Formatted phone number starts with +1 (3 points)

- Formatted phone number includes three digit area code between parentheses (3 points)
- Formatted phone number includes dash between first three and last four numbers of the 7 digit phone number (3 points)
- Spacing is followed (one space before and one space after the parentheses) (3 points)

Problem 2a: Cryptography

For this part you will be writing a series of functions that can be used as part of a "cryptography" program. Here are the functions you will be writing as well as some sample code that you can use to test your work.

Function #1

```
# function:  ascii_shift
# input:     a word to shift (String) and an amount to shift by (integer)
# processing: shifts each character in the supplied word to another position
#             in the ASCII table. the new position is dictated by the supplied
#             integer.  for example, if word = "apple" and num=1
#             the newly generated word would be:
#             bqpmf
#             because we added +1 to each character.
#
#             if we were to call the function with
#             word = "bqpmf" and num=-1 the newly generated word would be:
#             apple
#             because we added -1 to each character, which shifted
#             each character down by one position on the ASCII table.
#             in the event that an empty string is supplied no shift
#             will occur and an empty string should be returned
# output:    returns the newly generated word
```

Sample Program

```
word = "ABCDEFGH"
for i in range(-5, 6):
    print ("ASCII shifting", word, "by", i, "=>", ascii_shift(word, i))
```

Sample Output

```
ASCII shifting ABCDEFGH by -5 => <=>?@AB
```

```
ASCII shifting ABCDEFG by -4 => =>?@ABC
ASCII shifting ABCDEFG by -3 => >?@ABCD
ASCII shifting ABCDEFG by -2 => ?@ABCDE
ASCII shifting ABCDEFG by -1 => @ABCDEF
ASCII shifting ABCDEFG by 0 => ABCDEFG
ASCII shifting ABCDEFG by 1 => BCDEFGH
ASCII shifting ABCDEFG by 2 => CDEFGHI
ASCII shifting ABCDEFG by 3 => DEFGHIJ
ASCII shifting ABCDEFG by 4 => EFGHIJK
ASCII shifting ABCDEFG by 5 => FGHIJKL
```

Function #2

```
# function:  shift_right
# input:     a word to shift (String)
# processing: shifts all characters in the string to the right. the last char
#             acter in the string will be shifted to the beginning of
#             the string.  for example:
#             apple -> eappl
#             in the event that an empty string is supplied
#             no shift will occur and an empty string should be returned
# output:    returns the newly generated word
```

Sample Program

```
word = "hello world!"
print("Shifting right", word, "=>", shift_right(word))
```

Sample output

```
Shifting right hello world! => !hello world
```

Function #3

```
# function:  shift_left
# input:     a word to shift (String)
# processing: shifts all characters in the string to the left.
#
#           the first character in the string
#           will be shifted to the end of the string.  for example:
#           apple -> pplea
#
#           in the event that an empty string is supplied
#           no shift will occur and an empty string should be returned
# output:    returns the newly generated word
```

Sample Program

```
word = "hello world!"
print("Shifting left", word, "=>", shift_left(word))
```

Sample output

```
Shifting left hello world! => ello world!h
```

Function #4

```
# function:  flip
# input:     a word to flip (String)
# processing: flips the first half of the string with the second half of the
#
#           string. if the string has an even number of characters
#           this function will work as follows:
#           ABCD -> CDAB
#           if the string has an odd number of characters this function
#           will work as follows:
#           ABCDE -> DECAB
#           in the event that an empty string is supplied no flip
#           will occur and an empty string should be returned
# output:    returns the newly generated word
```

Sample Program

```
word = "ABCDEFGH"  
print ("Flipping", word, "=>", flip(word))  
  
word = "123456"  
print ("Flipping", word, "=>", flip(word))
```

Sample Output

```
Flipping ABCDEFG => EFGDABC  
Flipping 123456 => 456123
```

Function #5

```
# function:    add_letters  
# input:       a word to scramble (String) and a number of letters (integer)  
# processing:  adds a number of random letters (A-Z; a-z) after each letter  
#             in the supplied word. for example, if word="cat" and num=1  
#             we could generate any of the following:  
#             cZaQtR  
#             cwaRts  
#             cEaett  
#  
#             if word="cat" and num=2 we could generate any of the following:  
#             cRtaHFtui  
#             cnnaNYtjn  
#             czAaAitym  
#  
# output:      returns the newly generated word
```

Sample Program

```
# define original word  
original = "Hello!"
```

```
# loop to demonstrate the function
for num in range(1, 5):

    # scramble the word using 'num' extra characters
    scrambled = add_letters(original, num)

    # output
    print ("Adding", num, "random characters to", original, "->", scrambled)
```

Sample Output

```
Adding 1 random characters to Hello! -> HdeulHlHom!t
Adding 2 random characters to Hello! -> HTLedklFNljioMH!bi
Adding 3 random characters to Hello! -> HHHZeZrflqSflzOiosNU!jBk
Adding 4 random characters to Hello! -> HFtRKeivFlRNlUlGTaoOYwoH!JpXL
```

Hint: you will need to use a loop to generate the required # of random characters, and you will (obviously) need to use some random number functions as well. Think about the algorithm before you start coding! Draw out the steps you think you need to take on a piece of paper. For example: "Start with the first character in the source word. Then generate 'num' new random characters and concatenate these characters. Then move onto the next character in the source word and repeat the process".

Function #6

```
# function:    delete_characters
# input:       a word to analyze (String) and the number of characters
#              to remove (integer)
# processing:  the function starts at the first position in the supplied word
#              and keeps it. it then removes "num" characters from the word.
#              the process is repeated again if the word contains additional
#              characters – the next character is kept and "num" more
#              characters are removed. For example, if word="cZaYtU" and
#              num=1 the function will generate the following:
#              cat (keeping character 0, removing character 1, keeping
#              character 2, removing character 3, keeping character 4,
#              removing character 5)
```



```
# output:      returns the newly unscrambled word
```

Sample Program

```
word1 = "HdeulHlHom!t"
word2 = "HTLedklFNljioMH!bi"
word3 = "HHHZeZrflqSflz0iosNU!jBk"
word4 = "HFtRKeivFlRNlUlGTaooywoH!JpXL"

unscrambled1 = delete_characters(word1, 1)
print ("Removing 1 characer from", word1, "->", unscrambled1)

unscrambled2 = delete_characters(word2, 2)
print ("Removing 2 characers from", word2, "->", unscrambled2)

unscrambled3 = delete_characters(word3, 3)
print ("Removing 3 characers from", word3, "->", unscrambled3)

unscrambled4 = delete_characters(word4, 4)
print ("Removing 4 characers from", word4, "->", unscrambled4)
```

Sample Output

```
Removing 1 character from HdeulHlHom!t -> Hello!
Removing 2 character from HTLedklFNljioMH!bi -> Hello!
Removing 3 character from HHHZeZrflqSflz0iosNU!jBk -> Hello!
Removing 4 character from HFtRKeivFlRNlUlGTaooywoH!JpXL -> Hello!
```

Hint: String slicing may make your life a lot easier when writing this function!

This program should be named as follows: LastNameFirstName_assign7_part2a.py

Problem 2b

Now you are going to write an "encoder / decoder" program that makes use of your six cryptographic functions. Begin by writing a program that continually asks the user to enter in an encoding pattern and a word to encode. The user should also be able to end the program using a sentinel value.

If the user chooses to encode a word you should do the following:

- Ask the user for an "encoding pattern" string. This string will contain instructions on how to encode / decode a string. The valid commands that this string can contain are the following:
 - 'A' = add 1 character in between all characters
 - 'X' = delete 1 character in between all characters
 - 'F' = flip the string
 - 'U' = ASCII shift all characters by +1
 - 'D' = ASCII shift all characters by -1
 - 'L' = Shift all characters to the left by 1
 - 'R' = Shift all characters to the right by 1
 - Next, ask them to enter in a phrase that they want to encode.
 - Finally, apply the desired encoding pattern to their string, giving them feedback during the process. Apply the pattern one operation at a time, from left to right. Any invalid or unrecognized commands should be ignored.
 - Hint: use your functions!
 - Hint: you may need an accumulator variable to keep track of the current state of the word

Here are a few sample runnings of the program:

```
Enter an encoding pattern, 'end' to end: AFUUURRR
Enter a word to encode/decode: Hello World!
* Added 1 character: HYeflUlhoX FWLoorkladM!V
* Flipped: WLoorkladM!VHYeflUlhoX F
* ASCII shifted up: XMppslmbeN"WIZfgmVmipy!G
* ASCII shifted up: YNqqtmncf0#XJ[ghnWnjqz"H
* ASCII shifted up: Z0rrunodgP$YK\hioXokr{#I
* Shifted right: IZ0rrunodgP$YK\hioXokr{#
* Shifted right: #IZ0rrunodgP$YK\hioXokr{
* Shifted right: {#IZ0rrunodgP$YK\hioXokr

Enter an encoding pattern, 'end' to end: LLLDDDFX
Enter a word to encode/decode: {#IZ0rrunodgP$YK\hioXokr
* Shifted left: #IZ0rrunodgP$YK\hioXokr{
* Shifted left: IZ0rrunodgP$YK\hioXokr{#
* Shifted left: Z0rrunodgP$YK\hioXokr{#I
* ASCII shifted down: YNqqtmncf0#XJ[ghnWnjqz"H
* ASCII shifted down: XMppslmbeN"WIZfgmVmipy!G
* ASCII shifted down: WLoorkladM!VHYeflUlhoX F
* Flipped: HYeflUlhoX FWLoorkladM!V
```

```

* Deleted 1 character: Hello World!

Enter an encoding pattern, 'end' to end: end
Enter an encoding pattern, 'end' to end: FDDLILA
Enter a word to encode/decode: HappyBirthdaySadKitten
* Flipped: aySadKittenHappyBirthd
* ASCII shifted down: `xR`cJhssdmG`ooxAhqsgc
* ASCII shifted down: _wQ_bIgrrcLF_nnw@gprfb
* Shifted left: wQ_bIgrrcLF_nnw@gprfb_
* Shifted left: Q_bIgrrcLF_nnw@gprfb_w
* Shifted left: _bIgrrcLF_nnw@gprfb_wQ
* Added 1 character: _ibPIcgxrirtcQlvFs_Gnhnqwi@sgbpmrofnbM_VwEQH

Enter an encoding pattern, 'end' to end: XRRRUUF
Enter a word to encode/decode: _ibPIcgxrirtcQlvFs_Gnhnqwi@sgbpmrofnbM_VwEQH
* Deleted 1 character: _bIgrrcLF_nnw@gprfb_wQ
* Shifted right: Q_bIgrrcLF_nnw@gprfb_w
* Shifted right: wQ_bIgrrcLF_nnw@gprfb_
* Shifted right: _wQ_bIgrrcLF_nnw@gprfb
* ASCII shifted up: `xR`cJhssdmG`ooxAhqsgc
* ASCII shifted up: aySadKittenHappyBirthd
* Flipped: HappyBirthdaySadKitten

Enter an encoding pattern, 'end' to end: end

```

This program should be named as follows: LastNameFirstName_assign7_part2b.py

Problem 2 Points (57):

Test cases are provided here to help you debug your program. If you can't get the tests to pass, you can still receive partial credit for the program and functions.

- 2a – 3 points for each function, may use the following test cases and give partial credit if appropriate (24 points)

Input	Output
<code>print(ascii_shift("python",2))</code>	<code>r{vjqp</code>
<code>print(ascii_shift("python",-3))</code>	<code>mvqelk</code>
<code>print(shift_right("python"))</code>	<code>npytho</code>

<code>print(shift_left("python"))</code>	ythonp
<code>print(flip("python"))</code>	honpyt
<code>print(flip("1234567"))</code>	5674123
<code>print(add_letters("python",1))*</code>	pfryjztNJhhPoGvnEk
<code>print(delete_characters("python",2))</code>	ph

*Note: The add_letters function will randomly add letters, so the letters themselves don't matter, but the fact that a letter is added between every letter in the input string does.

- 2b (33 points)
 - Program continues until user types "end" (5 points)
 - Program posts intermediate decoding steps; one line per action taken (7 points)
 - Test the following cases; 7 points each:

Input Pattern	Input	Final Output
RRRULX	xgs@ghnimVHrraE@ttml OOB	PythonIsFun!
RUUUX	jIWjEmOrMqMb>oMpS`	computers
RDDDLX	eMpqtjIteJf{i}xhmWgleXpO	alphabetical

Problem 3: Identity Check – OPTIONAL Bonus Question

For this problem, you will be writing a program that prompts the user to enter a valid Social Security Number in the following format:

NNN-NN-NNNN

Where 'N' is a digit.

Further, the first three digits, called the "area number," cannot be equal to 000, 666, or between (and including) the numbers 900 and 999.

The middle two digits, called the "group number," must be an even number if the area number is between 001 and 499 and an odd number if the area number is between 500 and 899.

The program should display 'Valid SSN' if the supplied string is valid and 'Invalid SSN' as well as the following violated rules if they apply:

- Area number violated
- Group number violated
- Alphabetical characters included
- Missing hyphen(s)

For example:

What is your SSN? apple

Invalid SSN, try again! Only digits allowed. Missing hyphen(s).

What is your SSN? 423999888

Invalid SSN, try again! Missing hyphen(s).

What is your SSN? 000-45-6789

Invalid SSN, try again! Area number violated.

What is your SSN? 123-45-6789

Invalid SSN, try again! Group number violated.

What is your SSN? 123-46-6789

Valid SSN.

This program should be named as follows: LastNameFirstName_assign7_part3.py

Problem 3 Points (20):

- We will test 5 different SSNs worth 4 points each.